

```
1 INF = 999999
2
3 def shortest_distance(graph, u, v):
4     n = len(graph)
5
6     # Floyd-Warshall
7     for k in range(n):
8         for i in range(n):
9             for j in range(n):
10                 if graph[i][k] + graph[k][j] < graph[i][j]:
11                     graph[i][j] = graph[i][k] + graph[k][j]
12
13     return graph[u][v]
14
15
16 # Input
17 graph = [
18     [0, 3, INF, 7],
19     [8, 0, 2, INF],
20     [5, INF, 0, 1],
21     [2, INF, INF, 0]
22 ]
23
24 u = 0
25 v = 3
26
27 print(shortest_distance(graph, u, v))
```

```
1 def warshall(graph):
2     n = len(graph)
3
4     for k in range(n):
5         for i in range(n):
6             for j in range(n):
7                 if graph[i][k] == 1 and graph[k][j] == 1:
8                     graph[i][j] = 1
9     return graph
10
11 # Input
12 graph = [
13     [0, 1, 1],
14     [0, 0, 1],
15     [0, 0, 0]
16 ]
17 # Find transitive closure
18 graph = warshall(graph)
19
20 # Find vertex reachable from every other vertex
21 n = len(graph)
22 answer = -1
23
24 for j in range(n):
25     possible = True
26
27     for i in range(n):
28         if i != j and graph[i][j] == 0:
29             possible = False
30             break
31
32     if possible:
33         answer = j
34         break
35
36 print(answer)
```

2

...Program finished with exit code 0
Press ENTER to exit console.

```
1 def warshall(graph):
2     n = len(graph)
3     for k in range(n):
4         for i in range(n):
5             for j in range(n):
6                 if graph[i][k] == 1 and graph[k][j] == 1:
7                     graph[i][j] = 1
8     return graph
9 # Input
10 graph = [
11     [0, 1, 1, 0],
12     [0, 0, 1, 0],
13     [0, 0, 0, 1],
14     [0, 0, 0, 0]
15 ]
16 # Find transitive closure
17 graph = warshall(graph)
18
19 # Find vertex with maximum reachability
20 max_count = -1
21 answer = 0
22
23 for i in range(len(graph)):
24     count = 0
25
26     for j in range(len(graph)):
27         if i != j and graph[i][j] == 1:
28             count += 1
29
30     if count > max_count:
31         max_count = count
32         answer = i
33
34 print(answer)
```

```
1 INF = 999999
2
3 def floyd_warshall(graph):
4     n = len(graph)
5
6     for k in range(n):
7         for i in range(n):
8             for j in range(n):
9                 if graph[i][k] + graph[k][j] < graph[i][j]:
10                     graph[i][j] = graph[i][k] + graph[k][j]
11
12     return graph
13
14
15 # Input
16 graph = [
17     [0, 2, INF],
18     [2, 0, 4],
19     [INF, 4, 0]
20 ]
21
22 # Find shortest paths
23 graph = floyd_warshall(graph)
24
25 # Find diameter
26 diameter = 0
27
28 for i in range(len(graph)):
29     for j in range(len(graph)):
30         if graph[i][j] != INF:
31             diameter = max(diameter, graph[i][j])
32
33 print(diameter)
```

```

1 INF = 999999
2
3 def diameter(graph):
4     n = len(graph)
5
6     # Floyd-Warshall
7     for k in range(n):
8         for i in range(n):
9             for j in range(n):
10                if graph[i][k] + graph[k][j] < graph[i][j]:
11                    graph[i][j] = graph[i][k] + graph[k][j]
12
13    # Find maximum shortest-path distance
14    maximum = 0
15
16    for i in range(n):
17        for j in range(n):
18            if graph[i][j] != INF:
19                maximum = max(maximum, graph[i][j])
20
21    return maximum
22
23
24 # Input
25 graph = [
26     [0, 2, INF],
27     [2, 0, 3],
28     [INF, 3, 0]
29 ]
30
31 print(diameter(graph))

```

5

...Program finished with exit code 0
Press ENTER to exit console.


```
1 def warshall(graph):
2     n = len(graph)
3
4     for k in range(n):
5         for i in range(n):
6             for j in range(n):
7                 if graph[i][k] == 1 and graph[k][j] == 1:
8                     graph[i][j] = 1
9
10    return graph
11
12 # Input
13 graph = [
14     [0, 1, 0],
15     [0, 0, 1],
16     [1, 0, 0]
17 ]
18
19 # Find transitive closure
20 graph = warshall(graph)
21
22 # Check whether every vertex can reach every other vertex
23 n = len(graph)
24 strongly_connected = True
25
26 for i in range(n):
27     for j in range(n):
28         if i != j and graph[i][j] == 0:
29             strongly_connected = False
30
31 if strongly_connected:
32     print("Strongly Connected")
33 else:
34     print("Not Strongly Connected")
```

Strongly Connected

...Program finished with exit code 0
Press ENTER to exit console.

```
1 INF = 999999
2
3 graph = [
4     [0, 5, INF, 10],
5     [INF, 0, 3, INF],
6     [INF, INF, 0, 1],
7     [INF, INF, INF, 0]
8 ]
9
10 n = len(graph)
11
12 # Floyd-Warshall Algorithm
13 for k in range(n):
14     for i in range(n):
15         for j in range(n):
16             if graph[i][k] + graph[k][j] < graph[i][j]:
17                 graph[i][j] = graph[i][k] + graph[k][j]
18
19 # Print result
20 for row in graph:
21     print(row)
```



```
[999999, 0, 3, 4]
[999999, 999999, 0, 1]
[999999, 999999, 999999, 0]
```

```
..Program finished with exit code 0
Press ENTER to exit console.
```

```

1 def warshall(graph):
2     n = len(graph)
3
4     for k in range(n):
5         for i in range(n):
6             for j in range(n):
7                 if graph[i][k] == 1 and graph[k][j] == 1:
8                     graph[i][j] = 1
9
10    for row in graph:
11        print(row)
12
13
14 # Input
15 graph = [
16     [0, 1, 0, 0],
17     [0, 0, 1, 0],
18     [0, 0, 0, 1],
19     [0, 0, 0, 0]
20 ]
21
22 # Find transitive closure
23 warshall(graph)

```







```

0, 0, 1, 1]
0, 0, 0, 1]
0, 0, 0, 0]

```



```
1 INF = 999999
2 def findCity(n, edges, threshold):
3     dist = [[INF] * n for _ in range(n)]
4     for i in range(n):
5         dist[i][i] = 0
6     for u, v, w in edges:
7         dist[u][v] = w
8         dist[v][u] = w
9     for k in range(n):
10        for i in range(n):
11            for j in range(n):
12                if dist[i][k] + dist[k][j] < dist[i][j]:
13                    dist[i][j] = dist[i][k] + dist[k][j]
14    city = -1
15    min_count = INF
16    for i in range(n):
17        count = 0
18        for j in range(n):
19            if i != j and dist[i][j] <= threshold:
20                count += 1
21        if count <= min_count:
22            min_count = count
23            city = i
24    return city
25 n = 4
26 edges = [
27     [0, 1, 3],
28     [1, 2, 1],
29     [2, 3, 4],
30     [0, 3, 7]
31 ]
32 threshold = 4
33 print(findCity(n, edges, threshold))
```



3
...Program finished with exit code 0
Press ENTER to exit console.

```
1 def warshall(graph):
2     n = len(graph)
3
4     for k in range(n):
5         for i in range(n):
6             for j in range(n):
7                 if graph[i][k] == 1 and graph[k][j] == 1:
8                     graph[i][j] = 1
9
10    return graph
11
12
13 # Input
14 graph = [
15     [0, 1, 0],
16     [0, 0, 1],
17     [0, 0, 0]
18 ]
19
20 u = 0
21 v = 2
22
23 # Find reachability
24 graph = warshall(graph)
25
26 if graph[u][v] == 1:
27     print("Path Exists")
28 else:
29     print("Path Does Not Exist")
```

Path Exists

...Program finished with exit code 0
Press ENTER to exit console.

```

1 def warshall(graph):
2     n = len(graph)
3
4     for k in range(n):
5         for i in range(n):
6             for j in range(n):
7                 if graph[i][k] == 1 and graph[k][j] == 1:
8                     graph[i][j] = 1
9
10    return graph
11
12
13 # Input
14 graph = [
15     [0, 1, 0, 0],
16     [0, 0, 1, 0],
17     [0, 0, 0, 1],
18     [0, 0, 0, 0]
19 ]
20
21 source = 0
22
23 # Find transitive closure
24 graph = warshall(graph)
25
26 # Count reachable vertices
27 count = 0
28
29 for i in range(len(graph)):
30     if i != source and graph[source][i] == 1:
31         count += 1
32
33 print(count)

```

3

...Program finished with exit code 0
 Press ENTER to exit console.

```
1 def negative_cycle(vertices, edges):
2     dist = [0] * vertices
3
4     # Bellman-Ford
5     for i in range(vertices - 1):
6         for u, v, w in edges:
7             if dist[u] + w < dist[v]:
8                 dist[v] = dist[u] + w
9
10    # Check for negative weight cycle
11    for u, v, w in edges:
12        if dist[u] + w < dist[v]:
13            return True
14
15    return False
16
17
18 # Input
19 vertices = 4
20
21 edges = [
22     (0, 1, 1),
23     (1, 2, -1),
24     (2, 3, -1),
25     (3, 0, -1)
26 ]
27
28 if negative_cycle(vertices, edges):
29     print("Negative Weight Cycle Exists")
30 else:
31     print("No Negative Weight Cycle")
```



Negative Weight Cycle Exists

...Program finished with exit code 0
Press ENTER to exit console.

```

1 def cheapest_flight(n, flights, src, dst, K):
2     INF = 999999
3     dist = [INF] * n
4     dist[src] = 0
5
6     # At most K stops means K + 1 flights
7     for i in range(K + 1):
8         temp = dist.copy()
9
10        for u, v, cost in flights:
11            if dist[u] != INF:
12                temp[v] = min(temp[v], dist[u] + cost)
13
14        dist = temp
15
16        if dist[dst] == INF:
17            return -1
18        return dist[dst]
19
20 # Input
21 n = 4
22
23 flights = [
24     [0, 1, 100],
25     [1, 2, 100],
26     [2, 3, 100],
27     [0, 3, 500]
28 ]
29
30 src = 0
31 dst = 3
32 K = 1
33
34 print(cheapest_flight(n, flights, src, dst, K))

```

500

...Program finished with exit code 0
 Press ENTER to exit console.

main.py

```
1 def bellman_ford(V, edges, source):
2     dist = [999999] * V
3     dist[source] = 0
4     for i in range(V - 1):
5         for u, v, w in edges:
6             if dist[u] + w < dist[v]:
7                 dist[v] = dist[u] + w
8     for u, v, w in edges:
9         if dist[u] + w < dist[v]:
10            print("Negative Weight Cycle Detected")
11            return
12    print("Vertex Distance")
13    for i in range(V):
14        print(i, dist[i])
15 # Input
16 V = 5
17 edges = [
18     (0, 1, -1),
19     (0, 2, 4),
20     (1, 2, 3),
21     (1, 3, 2),
22     (1, 4, 2),
23     (3, 2, 5),
24     (3, 1, 1),
25     (4, 3, -3)
26 ]
27 source = 0
28 bellman_ford(V, edges, source)
```

2 2
3 -2
4 1

...Program finished with exit code 0
Press ENTER to exit console.

```
1 INF = 999999
2
3 def network_delay(times, n, k):
4     dist = [INF] * (n + 1)
5     dist[k] = 0
6
7     # Bellman-Ford
8     for i in range(n - 1):
9         for u, v, w in times:
10             if dist[u] != INF and dist[u] + w < dist[v]:
11                 dist[v] = dist[u] + w
12
13     # Check if all nodes are reachable
14     answer = 0
15
16     for i in range(1, n + 1):
17         if dist[i] == INF:
18             return -1
19         answer = max(answer, dist[i])
20
21     return answer
22
23
24 # Input
25 times = [
26     [2, 1, 1],
27     [2, 3, 1],
28     [3, 4, 1]
29 ]
30
31 n = 4
32 k = 2
33
34 print(network_delay(times, n, k))
```



2

...Program finished with exit code 0
Press ENTER to exit console.